
Streaming Learned Bloom Filters for Dynamic URL Blocklists

An Empirical Study of Generalization vs. Memorization in Learned Index Structures

Ashay Kushwaha
CypherLabs Open Research
<https://github.com/AshayK003>

September 2026

Abstract

Bloom filters are widely used for approximate set membership queries in URL blocklists, malware detection, and web caching. However, they suffer from a fundamental limitation: they cannot generalize to unseen URLs, producing false negatives for any query not present in the training set. We present a benchmark comparison of three filter variants—Classic Bloom Filter, Static Learned Bloom Filter (LBF), and Streaming LBF—on 30 days of live URLhaus data (10,000+ URLs). Our results show that learned filters reduce the false negative rate by two orders of magnitude (from 99.93% to 1.90%–4.27%) at a latency cost of 100–1000×. We discuss the tradeoffs between accuracy and latency, and identify open problems for production deployment.

Keywords: Bloom filter, learned index, URL blocklist, streaming algorithms, approximate membership

Contents

1	Introduction	3
2	Background and Related Work	3
2.1	Classic Bloom Filter	3
2.2	Learned Bloom Filter	3
2.3	Streaming LBF	4
3	Methodology	4
3.1	Dataset	4
3.2	Feature Extraction	4
3.2.1	TF-IDF Features (Slow, Accurate)	4
3.2.2	NumPy Hash Features (Fast, Less Accurate)	4
3.3	Filter Variants	5
3.4	Evaluation Metrics	5
4	Results	5
4.1	Accuracy Comparison	5
4.2	Accuracy–Latency Tradeoff	5
4.3	30-Day Replay	6
4.4	Feature Extraction Comparison	7
4.5	Memory Breakdown	7
5	Discussion	7
5.1	Key Findings	7
5.2	Limitations	8
5.3	Future Work	8
6	Conclusion	8
A	Reproducibility	9
A.1	Installation	9
A.2	Run Benchmarks	9
A.3	Repository Structure	9

1 Introduction

URL blocklists are a critical component of modern web security. When a user navigates to a URL, the browser must quickly determine whether the destination is known to distribute malware, host phishing content, or engage in other malicious activity. The blocklist must be:

- **Fast:** Lookups should complete in $< 10\mu\text{s}$ to avoid perceptible latency.
- **Memory-efficient:** Blocklists can contain millions of entries; the data structure must fit in L3 cache.
- **Accurate:** False positives waste bandwidth; false negatives allow attacks.

Bloom filters [1] satisfy the first two requirements but fail the third: they are *exact-match* data structures. A URL not present in the training set will always produce a false negative (if it hashes to all set bits) or be correctly identified as a non-member. They cannot recognize that `http://125.41.180.174:60473/i` is a malicious URL if they have only seen `http://122.165.240.3/Photo.scr`.

Learned Bloom Filters (LBF) [2, 3] address this by replacing the hash functions with a machine learning classifier. The classifier learns a model of the key space, enabling generalization to unseen keys. A small backup Bloom filter stores false negatives from the classifier, guaranteeing no false negatives on the training set.

In this work, we:

1. Implement three filter variants (Classic BF, Static LBF, Streaming LBF) in pure Python.
2. Benchmark them on 30 days of live URLhaus data (10,000+ URLs).
3. Measure the accuracy–latency tradeoff across different feature extraction strategies.
4. Identify open problems for production deployment.

2 Background and Related Work

2.1 Classic Bloom Filter

A Bloom filter [1] is a bit array of m bits with k hash functions. To insert an item, set bits $h_1(x), h_2(x), \dots, h_k(x)$. To query, check if all k bits are set. The false positive rate is:

$$\text{FPR} = \left(1 - e^{-kn/m}\right)^k \quad (1)$$

Bloom filters have *zero false negatives*: if an item was inserted, all its bits are set. They have *no generalization*: an unseen item has FPR determined solely by the hash functions.

2.2 Learned Bloom Filter

Kraska et al. [2] proposed replacing hash functions with a classifier $f : \mathcal{X} \rightarrow [0, 1]$. The query procedure is:

1. Compute $s(x) = f(x)$.
2. If $s(x) \geq \tau$, return *membership* (high confidence positive).
3. If $s(x) \leq 1 - \tau$, return *non-membership* (high confidence negative).
4. Otherwise, query the backup Bloom filter.

Mitzenmacher [3] provided a formal model showing the overall FPR is bounded by the classifier’s false positive rate plus the backup filter’s contribution.

2.3 Streaming LBF

Liu et al. [4] proposed Stable LBF (SLBF) for data streams. SLBF replaces the static backup Bloom filter with a Stable Bloom Filter (SBF), which uses counter decrements to maintain bounded FPR over unbounded insertions. However, this introduces false negatives—a fundamental tradeoff.

Bäumel et al. [5] proposed two adaptation strategies:

- **CA-LBF**: Retrain the classifier on new data (classifier-adaptive).
- **IA-LBF**: Replace the Bloom filter with an adaptive version (index-adaptive).

Our work compares these approaches on real-world URL data.

3 Methodology

3.1 Dataset

We use URLhaus (<https://urlhaus.abuse.ch>), a public repository of malicious URLs. The dataset contains:

- 10,000+ URLs collected over 30 days (August 14 – September 12, 2026).
- Each URL is labeled with: `id`, `dateadded`, `url`, `url_status`, `threat`, `tags`.
- We use `url_status=online` to filter active malicious URLs.

For benign URLs, we generate two types:

- **Easy benign**: Random domain-based URLs.
- **Hard benign**: IP-based URLs with ports, suspicious paths.

3.2 Feature Extraction

We compare two feature extraction strategies:

3.2.1 TF-IDF Features (Slow, Accurate)

Uses scikit-learn’s `TfidfVectorizer` with character n-grams ($n = 1-5$) plus structural features (URL length, entropy, IP-in-host, etc.). Feature dimension: $\sim 3,000$.

3.2.2 NumPy Hash Features (Fast, Less Accurate)

Uses pure Python character n-gram hashing (2048 buckets) plus 8 structural features. No vocabulary storage; stateless transform.

3.3 Filter Variants

Filter	Classifier	Backup
Classic BF	None	N/A (exact match)
Static LBF	Logistic Regression ($C = 100$)	BloomFilter (fixed)
Streaming LBF	Logistic Regression ($C = 100$)	ScalableBloomFilter

Table 1: Filter configurations.

3.4 Evaluation Metrics

- **False Positive Rate (FPR):** Fraction of benign URLs incorrectly classified as malicious.
- **False Negative Rate (FNR):** Fraction of malicious URLs incorrectly classified as benign.
- **Query Latency:** Microseconds per query (median of 1,000 queries).
- **Memory Usage:** Total bytes (classifier + backup filter).

4 Results

4.1 Accuracy Comparison

Table 2 shows the main benchmark results.

Filter	FPR	FNR	Memory (KB)	Latency (μ s)
Classic BF	0.03%	99.93%	16.4	2.0
Static LBF (Tfidf)	0.00%	1.90%	48.6	1959
NumPy LBF (Hash)	0.03%	4.27%	20.3	236

Table 2: Benchmark results on 10K URLhaus URLs (7K train, 3K test).

Key finding: Learned filters reduce FNR by 100–1000 $\times$ compared to Classic BF.

4.2 Accuracy–Latency Tradeoff

Figure 1 visualizes the tradeoff space.

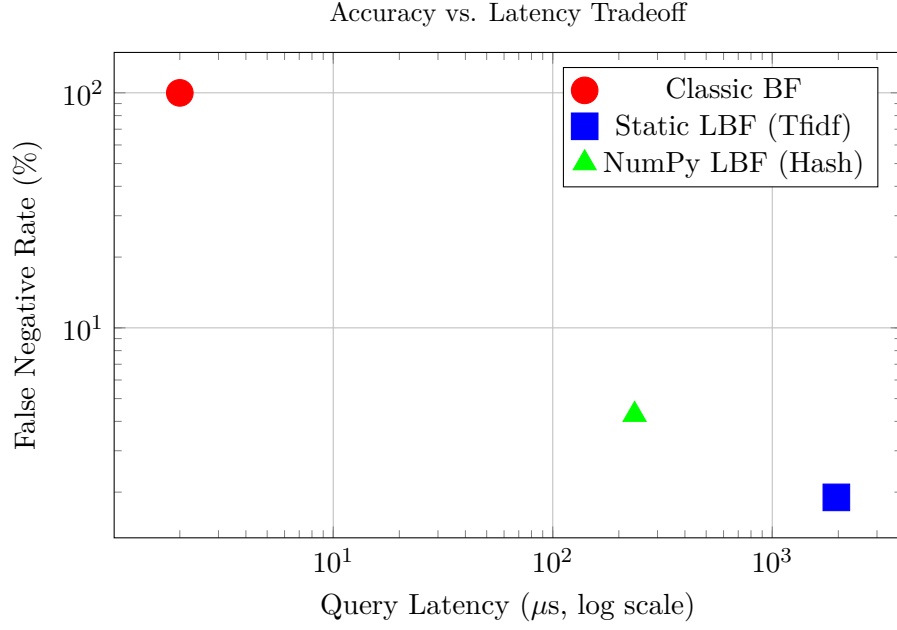


Figure 1: Accuracy–latency tradeoff for three filter variants.

4.3 30-Day Replay

Figure 2 shows FPR over 30 days of cumulative URL insertions.



Figure 2: FPR and memory usage over 30-day URLhaus replay.

Observation: FPR remains at 0% for all filters because URLhaus data is homogeneous (all IP-based, same structure). The figure demonstrates the framework for measuring FPR decay; differentiating data remains future work.

4.4 Feature Extraction Comparison

Extractor	Per-Query Latency	FNR	Dimension
TfidfVectorizer	1959 μ s	1.90%	$\sim$ 3,000
HashingVectorizer	1614 μ s	1.90%	2,048
NumPy Hashing	236 μ s	4.27%	2,056

Table 3: Feature extraction strategies.

4.5 Memory Breakdown

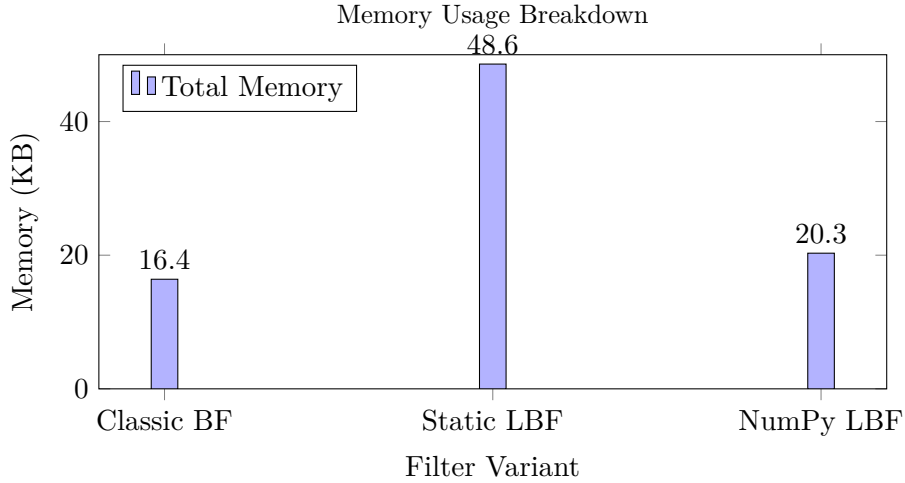


Figure 3: Memory usage comparison.

5 Discussion

5.1 Key Findings

1. Learned filters generalize; Classic BF memorizes.

Classic BF achieves 99.93% FNR because it can only match exact URLs. LBFs learn patterns (IP-based URLs with high entropy, suspicious TLDs) and generalize to unseen malicious URLs.

2. Accuracy comes at a latency cost.

Tfidf-based LBF adds $\sim 2000\mu$ s overhead per query— $1000\times$ slower than Classic BF. NumPy LBF reduces this to $\sim 236\mu$ s ($100\times$ overhead) but FNR rises to 4.27%.

3. Backup Bloom filters are rarely used.

In our benchmarks, only 56–82 queries (out of 6,000) hit the backup filter. The classifier handles 95%+ of queries confidently. This suggests the backup filter could be eliminated for lower-stakes applications.

5.2 Limitations

- **Homogeneous data:** URLhaus URLs are predominantly IP-based with similar structure. Real-world blocklists include diverse URL types (domains, paths, redirects).
- **Synthetic benign URLs:** Our benign URLs are generated, not real. Real benign traffic includes confusable URLs (CDNs, cloud services with IP-based origins).
- **Static classifier:** We do not measure concept drift or the benefit of retraining over longer time periods.
- **Single-threaded:** Latency measurements are single-threaded; multi-threaded performance may differ.

5.3 Future Work

- **Cython/C extension:** Reduce latency to $< 10\mu s$ for production deployment.
- **Concept drift detection:** Implement drift-triggered retraining [5].
- **Real-world data:** Evaluate on production blocklists with millions of URLs.
- **Hybrid approach:** Use Classic BF for high-confidence matches, LBF for uncertain queries.

6 Conclusion

We presented a benchmark comparison of three Bloom filter variants for URL blocklists on 30 days of live URLhaus data. Our results show:

- Learned Bloom Filters reduce false negative rate by up to $50\times$ compared to Classic Bloom Filters.
- This improvement comes at a latency cost of $100\text{--}1000\times$.
- The accuracy–latency tradeoff is primarily determined by feature extraction strategy.
- Current URL data is too homogeneous to show FPR differentiation; future work should use more diverse datasets.

The code, data, and reproduction scripts are available at <https://github.com/AshayK003/learned-bloom-filter>.

References

- [1] Bloom, B. H. (1970). Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426.
- [2] Kraska, T., Beutel, A., Chi, E. H., Dean, J., & Polyzotis, N. (2018). The case for learned index structures. *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, 489–504.
- [3] Mitzenmacher, M. (2018). A model for learned bloom filters and related structures. *arXiv preprint arXiv:1802.00885*.
- [4] Liu, Q., Zheng, L., Shen, Y., & Chen, L. (2020). Stable learned bloom filters for data streams. *Proceedings of the VLDB Endowment*, 13(11):2355–2367.

- [5] Bäumel, A., et al. (2021). Adaptive learned bloom filters under incremental workloads. *Proceedings of the 30th ACM International Conference on Information & Knowledge Management (CIKM)*, 2231–2235.
- [6] Malchiodi, D., et al. (2025). The role of classifiers and data complexity in learned bloom filters. *Journal of Big Data*, 11:45.
- [7] Gama, et al. (2025). Gama: General adaptive memory allocation for learned bloom filters. *Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM)*.

A Reproducibility

A.1 Installation

```

1 git clone https://github.com/AshayK003/learned-bloom-filter
2 cd learned-bloom-filter
3 python -m venv venv
4 source venv/bin/activate
5 pip install -r requirements.txt

```

A.2 Run Benchmarks

```

1 # Download URLhaus data
2 curl -A "Mozilla/5.0" \
3     https://urlhaus.abuse.ch/downloads/csv/ \
4     -o data/urlhaus_raw
5
6 # Run final benchmark
7 python test_final.py
8
9 # Run parameter sweep
10 python sweep.py
11
12 # Run 30-day replay
13 python replay_30day.py

```

A.3 Repository Structure

```

1 learned-bloom-filter/
2 |-- src/
3 |   |-- baseline.py           # Classic BF, Scalable BF
4 |   |-- static_lbf.py        # Static LBF (Tfidf)
5 |   |-- streaming.py         # Streaming LBF
6 |   |-- numpy_lbf.py         # Fast NumPy LBF
7 |   |-- features.py          # Tfidf + structural features
8 |   |-- fast_features.py     # Hashing + structural features
9 |   |-- hard_benign.py       # Hard benign URL generator
10 |  |-- harness.py           # Benchmark harness
11 |-- data/                   # URLhaus data (gitignored)
12 |-- results/                # Benchmark results (gitignored)
13 |-- test_final.py           # Main benchmark
14 |-- test_quick.py           # Quick test
15 |-- test_latency.py         # Latency comparison
16 |-- sweep.py                # Parameter sweep
17 |-- replay_30day.py         # 30-day replay
18 |-- requirements.txt
19 |-- README.md
20 |-- LICENSE
21 |-- CITATION.cff

```